

第25讲：LLM 到底有没有“记忆”？

参数记忆、Context、KV Cache、RAG 与 Agent Memory

本讲目标
把所有常被叫作“记忆”的机制拆开。学完后你应该能回答：哪些信息写进了模型参数？哪些只在当前上下文里？KV Cache 为什么不是长期记忆？RAG 为什么更像外部存储？一个真正的 Agent Memory 系统怎样写入、检索、更新和遗忘？

参数记忆 / Parametric Memory	模型权重中的分布式模式
Context / Working Context	本轮请求真正摆在模型面前的 token
KV Cache	对当前上下文已经算过的 K/V 的计算缓存
External Memory / RAG	数据库、文件、向量库等外部持久存储
Agent Memory	Runtime 负责写入、检索、更新、遗忘的系统级记忆

都可以被口语上叫“记忆”，但它们的存储位置、生命周期和作用完全不同。

先记住一句：“LLM 有记忆吗？”这个问题没有一个简单的 Yes / No。关键是先问：你说的是哪一种记忆？存在哪里？能保存多久？怎样被读出来？

1. 为什么“记忆”这个词特别容易把人绕晕？

因为从用户视角看，只要系统能够“记住以前的信息”，我们就会自然地叫它记忆。但在 LLM 系统内部，至少存在五种完全不同的机制：参数、Context、KV Cache、外部检索存储、Agent Runtime Memory。

机制	存在哪里	生命周期	怎样读出	是否真正修改模型参数
参数记忆	模型权重	长期，直到重新训练/替换模型	通过前向推理隐式调用	是
Context	当前输入 token	当前请求/会话的一部分	Attention 直接读取	否
KV Cache	推理显存/内存	当前推理序列	Attention 复用 K/V	否
RAG / 外部存储	数据库/文件/向量库	可长期持久化	先检索，再注入 Context	否
Agent Memory	Runtime 管理的外部状态	可跨轮次/跨任务	按策略检索并注入	通常否

关键
如果不先区分这些层次，就会出现典型误解：把 KV Cache 当长期记忆、把聊天历史当模型权重变化、把向量数据库当成模型已经“学会”了知识。

2. 参数记忆：知识真的“写进参数”了吗？

预训练和微调会通过梯度更新改变模型参数。因此模型确实会在权重中形成对语言、事实、模式和技能的分布式表示。很多论文把这种现象称作 parametric knowledge / parametric memory。
但这里的“记忆”不是一张“问题 -> 答案”的数据库表。一个事实通常分散地依赖大量参数与隐藏表示，并且不同事实、语法、概念之间高度纠缠。

通俗比喻
参数记忆更像“长期练出来的能力和直觉”，而不是“口袋里的一张便签”。你会骑自行车，不代表大脑里有一行 SQL 记录：bike_skill = true。

- 这解释了两个现象：
- ① 模型可以泛化：没有逐字见过的问题也可能回答；
 - ② 模型也可能记错或混淆：因为它不是在执行精确数据库查询。

重要限制
Prompt 里告诉模型一个新事实，并不会自动写回模型权重。Context 中的信息只对当前推理起作用；如果要真正改变参数，通常需要继续训练、微调、编辑模型或其他参数更新机制。

3. Context：最像“工作记忆”的东西

Context 是这一轮推理真正提供给模型的 token。它可以包含 System Prompt、当前问题、历史对话、RAG 文档、工具结果、Memory 检索结果等。
在第21讲里我们已经用过一个比喻：

Context = 当前工作台。 你现在把什么摆到桌面上，模型现在就能看什么。

Context 的优点是立即可用，不需要修改参数；缺点是容量有限，而且信息会随着窗口截断、压缩、摘要或会话结束而消失。

问题	Context 的回答
能马上加入新事实吗？	可以
需要重新训练模型吗？	不需要
会永久改变模型吗？	不会
窗口满了怎么办？	截断、压缩、摘要、检索
是否等于长期记忆？	不等于

4. KV Cache：它为什么绝对不能等同于长期记忆？

KV Cache 是推理优化机制。模型在 Prefill / Decode 时已经为过去 token 计算过 Key 和 Value，就把它们暂存起来，后续 token 不必重新计算。

它保存的是当前序列的中间计算结果，而不是一个面向未来任务设计的知识库。

KV Cache	Agent Memory
存的是 K/V 中间张量	存的是可解释的事件/事实/摘要/状态
目标是减少重复计算	目标是跨时间保留有用信息
通常随请求/会话结束释放	可跨轮次、跨任务持久化
由模型推理引擎自动维护	由 Runtime 的记忆策略管理
不能主动做语义检索	通常需要检索与排序

一句话

KV Cache 是“计算缓存”，不是“人生经历”。

5. RAG / 外部存储：把“记忆”放到模型外面

如果我们希望系统长期保存大量事实、文档或历史记录，最自然的方法不是把它们全塞进模型参数，而是存到模型外部：数据库、文件系统、搜索引擎、向量数据库、知识图谱等。

需要时：

Query -> Retrieval -> Relevant Memory -> Context -> LLM

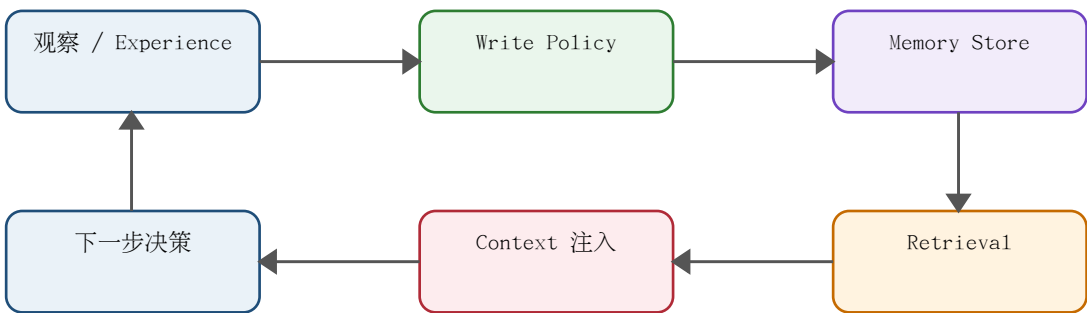
这就是 RAG / External Memory 的基本思路。

重要区别

外部存储中的内容并没有自动变成“参数知识”。只有当它被检索并注入当前 Context 后，模型这一轮才真正能使用。

因此，RAG 的强弱取决于至少四件事：存什么、怎么索引、怎么检索、怎么把结果放进 Context。

6. Agent Memory：真正需要设计的是一个生命周期



Agent Memory 的关键不是“有个数据库”，而是写入、检索、更新和遗忘策略。

一个成熟的 Agent Memory 系统不是“接个向量数据库”就结束。它至少包含：

阶段	核心问题
Observe / Experience	发生了什么？哪些信息值得考虑？
Write	哪些内容值得保存？保存原文还是摘要？
Store	保存在哪里？文本、结构化字段、向量、图？
Retrieve	当前任务需要哪几条历史？
Inject	检索结果怎样进入 Context？
Update	新信息与旧信息冲突时怎么办？
Forget / Compact	哪些内容应该删除、降权或摘要？

核心

记忆系统真正的智能，往往不在 Store，而在 Write Policy + Retrieval Policy + Update/Forget Policy。

7. Agent Memory 常见分类: Working / Episodic / Semantic / Procedural

类型	通俗理解	例子
Working Memory	当前任务临时状态	当前计划、未完成步骤、临时变量
Episodic Memory	“发生过什么”	上周某任务失败, 因为 API 限流
Semantic Memory	“我知道什么”	项目数据库字段含义、稳定事实
Procedural Memory	“我通常怎么做”	遇到报错先看日志, 再复现最小案例
Profile / Preference	长期偏好与约束	回答格式、工具偏好、权限限制

这些分类来自认知科学/Agent 设计中的类比, 不同系统命名会不同。做论文时不要只报一个 “memory enabled”, 而应该明确到底保存什么、如何读写。

8. Write Policy: 什么都记, 反而可能最差

一个 Agent 每轮都会产生大量内容: 用户输入、计划、工具调用、工具结果、草稿、反思、失败日志。如果全部永久写入 Memory, 结果往往是:

Memory Bloat -> Retrieval Noise -> Context Bloat -> More Cost / More Errors

因此 Write Policy 通常要考虑:

维度	问题
重要性	这条信息未来真的可能有用吗?
新颖性	它是不是已经存过了?
稳定性	这是长期事实, 还是暂时状态?
可信度	来源是否可靠? 是否经过验证?
隐私/权限	是否允许长期保存?
粒度	保存原始日志、结构化字段还是摘要?

风险

如果一个 Agent 把未经验证的幻觉写入长期 Memory, 后续检索会把错误再次注入 Context, 形成 “错误自我强化”。

9. Retrieval Policy: 找到 “相关” 不等于找到 “该用的”

最简单的向量检索通常按语义相似度排序。但真实 Agent Memory 常常还需要把其他信号一起考虑。一个教学性的打分可以写成:

$\text{score} = a * \text{similarity} + b * \text{recency} + c * \text{importance} + d * \text{reliability}$
--

也就是说, 不仅要问 “像不像”, 还要问: 是不是最近的? 是不是重要? 是不是可信?

信号	作用	潜在副作用
Similarity	找到语义相关内容	可能召回旧但相似的错误事实

信号	作用	潜在副作用
Recency	偏好最近经历	可能忘掉长期稳定规律
Importance	优先关键事件	重要性评分本身可能不准
Reliability	优先已验证来源	需要额外的来源追踪

10. 更新与冲突：旧记忆和新事实打架怎么办？

长期系统一定会遇到冲突：昨天的事实今天可能已经变了；早期 Memory 可能是错的；两个来源可能不同。

如果 Memory Store 只是“不断追加”，就会形成多个互相矛盾的版本。

常见策略包括：

策略	做法
Overwrite	新值直接覆盖旧值
Versioning	保留版本 + 时间戳
Confidence Update	新证据调整置信度
Source-aware	记录来源，优先更可靠/更新来源
Conflict Set	保留冲突，交给 verifier 或人工决策

工程原则

对会变化的事实，最好让 Memory 带时间戳和来源，而不是只保存一句没有上下文的“真相”。

11. Forgetting / Compaction: 遗忘不是缺陷，可能是能力

无限增长的 Memory 会带来存储、检索、成本和错误积累问题。因此长期 Agent 必须有某种遗忘或压缩机制。

常见做法:

方法	作用
TTL / Expiration	临时状态到期自动失效
Recency Decay	越久远权重越低
Summarization	多条经历压缩成摘要
Deduplication	删除重复记录
Consolidation	从多次 episode 提炼稳定规律
Archival	冷数据仍保存，但默认不进入检索

直觉
好的 Memory
不是“什么都不忘”，而是“该留下的长期留下，该降权的逐步降权，该淘汰的及时淘汰”。

12. Parameter Memory 与 Agent Memory 的本质区别

维度	参数记忆	Agent Memory
更新方式	训练/微调/模型编辑	Runtime 直接写数据库/文件
更新速度	慢、成本高	快、可在线更新
可解释性	通常较弱、分布式	可以存原始文本/结构化记录
检索方式	隐式前向推理	显式搜索/检索/排序
冲突处理	难精确定位	可做版本、来源、时间戳
持久性	跟随模型 checkpoint	跟随外部存储

因此长期 Agent 更常见的路线是：稳定能力放在模型参数里，动态经历放在外部 Memory 里。

13. Memory ≠ Learning: 存下来与真正学会是两件事

Agent 把一次失败记录写进数据库，并不等于模型参数已经发生改变。

下一次任务如果检索到这条经验，模型可以暂时利用它；但如果检索不到，模型可能又犯同样错误。

例子
“上次调用接口时需要先设置 timeout=30” 被写入 Memory。下一次如果 Retriever 取到它，Agent 会表现得像‘学会了’；如果没取到，底层模型并没有因此永久掌握。

这也是研究 Persistent Agent 时必须区分的两个层次:

Experience stored externally != Capability internalized into weights

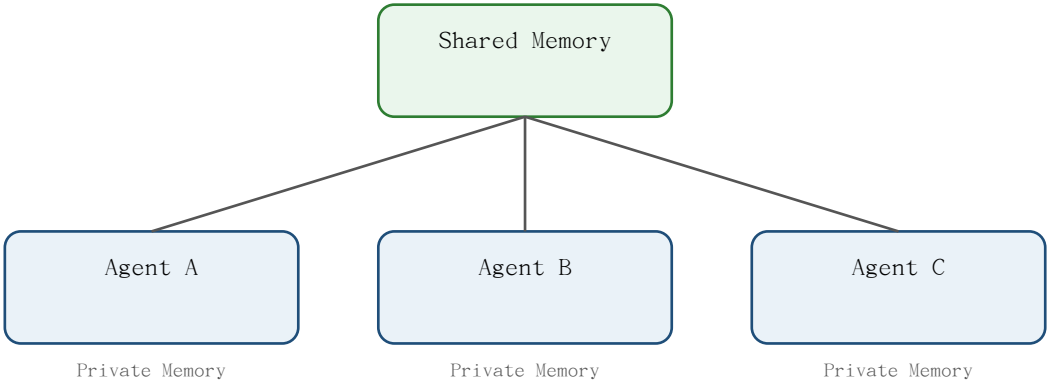
14. Memory 失败模式：不是“记住”就一定更可靠

失败模式	现象
False Memory	错误/幻觉被写入
Stale Memory	过去正确，现在已经过期
Retrieval Miss	有正确记忆，但没检索出来
Retrieval Pollution	大量相似但无关内容挤掉关键记录
Context Overload	检索太多，反而降低利用能力
Memory Poisoning	恶意内容进入长期存储
Privacy Leakage	不该保留的信息被长期保存/跨任务使用
Over-consolidation	摘要过度，丢掉关键细节

安全视角
Agent Memory 扩大了攻击面：攻击者不一定要让模型当场执行错误动作，也可以尝试把恶意信息“种进记忆”，等待以后被检索。

15. 共享记忆 vs 私有记忆：多 Agent 系统的关键设计

共享提高信息传播，但也可能同步错误；私有记忆保留差异，但也可能造成知识不对称。



多 Agent 系统可以让所有 Agent 共享同一个 Memory，也可以给每个 Agent 独立 Memory，还可以采用混合结构。

结构	优点	风险
全部共享	信息传播快、重复劳动少	错误同步、群体同质化
全部私有	保留差异、并行探索	知识孤岛、重复成本
共享 + 私有	兼顾全局和个体经验	设计与同步更复杂

16. 与你的研究问题连接：经历如何形成 Agent 异质性？

如果不同 Agent 长期经历不同任务、工具反馈、失败案例和环境，它们的 Memory Store 会逐渐分化。

Experience Divergence -> Memory Divergence -> Retrieval Context Divergence -> Behavior / Error Divergence

这提供了一条非常自然的研究路径:

变量	可以怎么测
Experience Divergence	不同 Agent 接触到的事件/任务差异
Memory Divergence	记忆条目、主题、向量分布差异
Behavior Divergence	策略/答案/工具选择差异
Error Correlation	是否在同样样本上一起错
Collective Gain	聚合后相对单 Agent 的收益

科研洞察
真正有意义的“长期异质性”不只是换 persona 或 random seed，而是不同经历通过 Memory / Retrieval 持续改变 Agent 未来可见的信息，从而形成稳定的行为与错误结构差异。

17. 但要小心: Memory Divergence 不自动等于 Collective Intelligence

如果不同 Agent 只是各自记住了不同噪声，它们确实会表现不同，但这种差异未必有用。

理想情况是:

Useful Diversity -> Lower Error Correlation -> Better Aggregation

而不是:

Random Noise -> Different Answers -> No Stable Collective Gain

实验原则
研究异质性时，不要只测答案多样性 (diversity)。还要测：差异是否稳定、是否和经历因果相关、是否降低错误相关性、是否提高 compute-matched collective gain。

18. 一个可复现的 Memory 实验应该记录什么？

层	至少记录
Experience	Agent 看到了什么事件/反馈
Write	哪些内容被写入，为什么写
Memory Store	条目、时间戳、来源、可信度
Retrieval	查询是什么、召回了哪些条目、分数多少
Context	最终哪些 Memory 真正注入 Prompt
Action	模型输出/工具调用是什么
Outcome	任务是否成功、错误类型是什么

最低闭环
只报告“开启了 memory”远远不够。你必须能够从最终错误反查：是没写进去？写错了？没检索到？检索了错误条目？还是模型拿到正确记忆后仍然没用？

19. 常见误区汇总

误区	更准确的理解
聊天越久，模型参数就自动更新	通常没有；更多是 Context/外部 Memory 在变化
KV Cache 就是模型记忆	它是推理计算缓存
向量数据库连上以后模型就学会了	只有检索结果进入 Context 后本轮才可用
Memory 越大越好	会产生检索噪声、成本和错误积累
不同 Agent 有不同 Memory 就一定更强	差异可能只是噪声
记住一次失败 = 永久学会	取决于未来是否能正确检索和利用
摘要越短越好	过度压缩会丢失关键因果细节

20. 小实验：一个极小的 Agent Memory Store

材料包中的 Python 实验实现一个不依赖第三方库的 Memory Store。每条 Memory 包含：

`text + timestamp + importance + reliability`

检索分数采用一个教学公式：

`score = 0.55*similarity + 0.20*recency + 0.15*importance + 0.10*reliability`

实验会演示三个现象：

现象	你会看到什么
相关记忆检索	与 timeout 相关的经历会被优先召回
过期/错误记忆	旧错误事实仍可能因为语义相似被召回
可靠性权重	经过验证的记录更容易压过低可信记录

注意

这个打分公式只是教学示例，不是任何标准算法。真实 Agent Memory 可能使用 embedding、BM25、reranker、规则、时间衰减、来源可信度等多种机制。

21. 练习题

- 1) 参数记忆和 Context 最根本的区别是什么？
- 2) 为什么 KV Cache 不应该被叫作长期记忆？
- 3) RAG 为什么可以看作“外部记忆”，但它又不等于 Agent 已经学会了知识？
- 4) 一个 Agent Memory 系统为什么必须设计 Write Policy？
- 5) 为什么 Retrieval 只按语义相似度可能不够？
- 6) 什么叫 Stale Memory？怎样缓解？
- 7) 共享 Memory 为什么可能降低多 Agent 的功能性异质性？
- 8) 如果不同 Agent 长期经历不同，如何验证“经历真的导致了错误去相关”？
- 9) 设计一个实验，区分“Memory 多样性”与“随机噪声多样性”。
- 10) 为什么论文里只写“we enable memory”是不够的？

答案见材料包《03_第25讲_练习题与答案.pdf》。

22. 本讲总结

一句话总纲

LLM 的“记忆”不是单一模块：参数记忆是训练得到的长期分布式能力；Context 是当前工作台；KV Cache 是当前推理的计算缓存；RAG 是外部取证；Agent Memory 则是一套由 Runtime 管理的“写入 - 存储 - 检索 - 更新 - 遗忘”系统。

机制	最短记法
Parametric Memory	练进权重里的长期模式
Context	这轮真正摆在模型面前的 token
KV Cache	过去 token 已算好的 K/V
RAG / External Memory	外部仓库，需要检索后才能用
Agent Memory	带策略的长期经历管理系统

下一讲：第26讲《RAG：为什么大语言模型需要“先查资料再回答”？——Embedding、向量检索、Chunk、Recall 与 Grounding》。